AGILE SOFTWARE DEVELOPMENT

Principles, Patterns, and Practices



Robert C. Martin

with contributions by James W. Newkirk and Robert S. Koss

Principles behind the Agile Manifesto

We follow these principles:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler
James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick
Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Agile Software Development

Principles, Patterns, and Practices

Robert Cecil Martin

Alan Apt Series



Pearson Education, Inc.
Upper Saddle River, New Jersey 07458

Library of Congress Cataloging-in-Publication Data

Martin, Robert C.
Agile software development: principles, patterns, and practices / Robert Martin.
p. cm.
Includes bibliographical references and index.
ISBN 0-13-597444-5
1. Computer software—Development. 2. eXtreme programming. I. Title.

QA76.76.D47 M362 2002
005.1—dc21 2002070056

Vice President and Editorial Director, ECS: *Marcia J. Horton*
Publisher: *Alan R Apt*
Assistant Editor: *Toni D. Holm*
Editorial Assistant: *Patrick Lindner*
Vice President and Director of Production and Manufacturing, ESM: *David W. Riccardi*
Executive Managing Editor: *Vince O'Brien*
Assistant Managing Editor: *Camille Trentacoste*
Production Editor: *Fran Daniele*
Director of Creative Services: *Paul Belfanti*
Creative Director: *Carole Anson*
Cover Designer: *Bruce Kenselaar*
Art Editor: *Greg Dulles*
Manufacturing Manager: *Trudy Pisciotti*
Manufacturing Buyer: *Lynda Castillo*
Marketing Manager: *Pamela Shaffer*
Assistant Marketing Manager: *Barrie Reinhold*



© 2003 by Pearson Education, Inc.
Pearson Education, Inc.
Upper Saddle River, NJ 07458

The author and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of the theories and programs to determine their effectiveness. The author and publisher shall not be liable in any event for incidental or consequential damages in connection with, or arising out of, the furnishing, performance, or use of these programs.

All rights reserved. No part of this book may be reproduced, in any form or by any means, without permission in writing from the publisher.

Printed in the United States of America

20 19 18 17 16 15 14

ISBN 0-13-597444-5

Pearson Education LTD., *London*
Pearson Education Australia PTY, Limited, *Sydney*
Pearson Education Singapore, Pte. Ltd.
Pearson Education North Asia Ltd., *Hong Kong*
Pearson Education Canada, Ltd., *Toronto*
Pearson Educación de Mexico, S.A. de C.V.
Pearson Education—Japan, *Tokyo*
Pearson Education Malaysia, Pte. Ltd.
Pearson Education, Upper Saddle River, *New Jersey*

Foreword

I'm writing this foreword right after having shipped a major release of the Eclipse open source project. I'm still in recovery mode, and my mind is bleary. But one thing remains clearer than ever: that people, not processes, are the key to shipping a product. Our recipe for success is simple: work with individuals obsessed with shipping software, develop with lightweight processes that are tuned to each team, and adapt constantly.

Double-clicking on developers from our teams reveals individuals who consider programming the focus of development. Not only do they write code; they digest it constantly to maintain an understanding of the system. Validating designs with code provides feedback that's crucial for getting confidence in a design. At the same time, our developers understand the importance of patterns, refactoring, testing, incremental delivery, frequent builds, and other best-practices of XP that have altered the way we view methodologies today.

Skill in this style of development is a prerequisite for success in projects with high technical risk and changing requirements. Agile development is low-key on ceremony and project documentation, but it's intense when it comes to the day-to-day development practices that count. Putting these practices to work is the focus of this book.

Robert is a longtime activist in the object-oriented community, with contributions to C++ practice, design patterns, and object-oriented design principles in general. He was an early and vocal advocate of XP and agile methods. This book builds on these contributions, covering the full spectrum of agile development practice. It's an ambitious effort. Robert makes it more so by demonstrating everything through case studies and lots of code, as befits agile practice. He explains programming and design by actually doing it.

This book is crammed with sensible advice for software development. It's equally good whether you want to become an agile developer or improve the skills you already have. I was looking forward to this book, and I wasn't disappointed.

Erich Gamma
Object Technology International

For Ann Marie, Angela, Micah, Gina, Justin, Angelique, Matt, and Alexis . . .

There is no greater treasure,
Nor any wealthier trove,
Than the company of my family,
And the comfort of their love.

Preface



But Bob, you said you'd be done with the book last year.

—Claudia Frers, *UML World*, 1999

Agile development is the ability to develop software quickly, in the face of rapidly changing requirements. In order to achieve this agility, we need to employ practices that provide the necessary discipline and feedback. We need to employ design principles that keep our software flexible and maintainable, and we need to know the design patterns that have been shown to balance those principles for specific problems. This book is an attempt to knit all three of these concepts together into a functioning whole.

This book describes those principles, patterns, and practices and then demonstrates, how they are applied by walking through dozens of different case studies. More importantly, the case studies are not presented as complete works. Rather, they are designs *in progress*. You will see the designers make mistakes, and you will observe how they identify the mistakes and eventually correct them. You will see them puzzle over conundrums and worry over ambiguities and trade-offs. You will see the *act* of design.

The Devil Is in the Details



This book contains a *lot* of Java and C++ code. I hope you will carefully read that code since, to a large degree, the code is the *point* of the book. The code is the actualization of what this book has to say.

There is a repeating pattern to this book. It consists of a series of case studies of varying sizes. Some are very small, and some require several chapters to describe. Each case study is preceded by material that is meant to prepare you for it. For example, the Payroll case study is preceded by chapters describing the object-oriented design principles and patterns used in the case study.

The book begins with a discussion of development practices and processes. That discussion is punctuated by a number of small case studies and examples. From there, the book moves on to the topic of design and design principles, and then to some design patterns, more design principles that govern packages, and more patterns. All of these topics are accompanied by case studies.

So prepare yourself to read some code and to pore over some UML diagrams. The book you are about to read is *very* technical, and its lessons, like the devil, are in the details.

A Little History

Over six years ago, I wrote a book entitled *Designing Object-Oriented C++ Applications using the Booch Method.* It was something of magnum opus for me, and I was very pleased with the result and with the sales.

This book started out as a second edition to *Designing*, but that's not how it turned out. Very little remains of the original book in these pages. Little more than three chapters have been carried through, and those chapters have been massively changed. The intent, spirit, and many of the lessons of the book are the same. And yet, I've learned a tremendous amount about software design and development in the six years since *Designing* came out. This book reflects that learning.

What a half-decade! *Designing* came out just before the Internet collided with the planet. Since then, the number of abbreviations we have to deal with has doubled. We have Design Patterns, Java, EJB, RMI, J2EE, XML, XSLT, HTML, ASP, JSP, Servlets, Application Servers, ZOPE, SOAP, C#, .NET, etc., etc. Let me tell you, it's been hard to keep the chapters of this book reasonably current!

The Booch Connection

In 1997, I was approached by Grady Booch to help write the third edition of his amazingly successful *Object-Oriented Analysis and Design with Applications*. I had worked with Grady before on some projects, and I had been an avid reader and contributor to his various works, including UML. So I accepted with glee. I asked my good friend Jim Newkirk to help out with the project.

Over the next two years, Jim and I wrote a number of chapters for the Booch book. Of course, that effort meant that I could not put as much effort into this book as I would have liked, but I felt that the Booch book was worth the contribution. Besides, this book was really just a second edition of *Designing* at the time, and my heart wasn't in it. If I was going to say something, I wanted to say something new and different.

Unfortunately, that version of the Booch book was not to be. It is hard to find the time to write a book during normal times. During the heady days of the ".com" bubble, it was nearly impossible. Grady got ever busier with Rational and with new ventures like Catapulse. So the project stalled. Eventually, I asked Grady and Addison–Wesley if I could have the chapters that Jim and I wrote to include in *this* book. They graciously agreed. So several of the case study and UML chapters came from that source.

The Impact of Extreme Programming

In late 1998, XP reared its head and challenged our cherished beliefs about software development. Should we create lots of UML diagrams prior to writing any code, or should we eschew any kind of diagrams and just write lots of code? Should we write lots of narrative documents that describe our design, or should we try to make the *code* narrative and expressive so that ancillary documents aren't necessary? Should we program in pairs? Should we write tests before we write production code? What should we do?

This revolution came at an opportune time for me. During the middle to late 90s, Object Mentor was helping quite a few companies with object-oriented (OO) design and project management issues. We were helping companies get their projects *done*. As part of that help, we instilled our own attitudes and practices into the teams. Unfortunately, these attitudes and practices were not written down. Rather, they were an oral tradition that was passed from us to our customers.

By 1998, I realized that we needed to write down our process and practices so that we could better articulate them to our customers. So, I wrote many articles about process in the *C++ Report*.[1] These articles missed the mark. They were informative, and in some cases entertaining, but instead of codifying the practices and attitudes

1. These articles are available in the "publications" section of `http://www.objectmentor.com`. There are four of them. The first three are entitled "Iterative and Incremental Development" (I, II, III). The last is entitled "C.O.D.E Culled Object Development procEss."

that we actually used in our projects, they were an unwitting compromise to values that had been imposed upon me for decades. It took Kent Beck to show me that.

The Beck Connection

In late 1998, as I was fretting over codifying the Object-Mentor process, I ran into Kent's work on Extreme Programming (XP). The work was scattered through Ward Cunningham's *wiki*[2] and was mixed with the writings of many others. Still, with some work and diligence I was able to get the gist of what Kent was talking about. I was intrigued, but skeptical. Some of the things that XP talked about were exactly on target for my concept of a development process. Other things, however, like the lack of an articulated design step, left me puzzled.

Kent and I could not have come from more disparate software circumstances. He was a recognized Smalltalk consultant, and I was a recognized C++ consultant. Those two worlds found it difficult to communicate with one another. There was an almost Kuhnian[3] paradigm gulf between them.

Under other circumstances, I would never have asked Kent to write an article for the *C++ Report*. But the congruence of our thinking about process was able to breech the language gulf. In February of 1999, I met Kent in Munich at the OOP conference. He was giving a talk on XP in the room across from where I was giving a talk on principles of OOD. Being unable to hear that talk, I sought Kent out at lunch. We talked about XP, and I asked him to write an article for the *C++ Report*. It was a great article about an incident in which Kent and a coworker had been able to make a sweeping design change in a live system in a matter of an hour or so.

Over the next several months, I went through the slow process of sorting out my own fears about XP. My greatest fear was in adopting a process in which there is no explicit up-front design step. I found myself balking at that. Didn't I have an obligation to my clients, and to the industry as a whole, to teach them that design is important enough to spend time on?

Eventually, I realized that I did not really practice such a step myself. Even in all the articles and books I had written about design, Booch diagrams, and UML diagrams, I had always used code as a way to verify that the diagrams were meaningful. In all my customer consulting, I would spend an hour or two helping them to draw diagrams and then I would direct them to explore those diagrams with code. I came to understand that though XP's words about design were foreign (in a Kuhnian[4] sense), the practices behind the words were familiar to me.

My other fears about XP were easier to deal with. I had always been a closet pair programmer. XP gave me a way to come out of the closet and revel in my desire to program with a partner. Refactoring, continuous integration, and customer on-site were all very easy for me to accept. They were very close to the way I already advised my customers to work.

One practice of XP was a revelation for me. Test-first design sounds innocuous when you first hear it. It says to write test cases before you write production code. All production code is written to make failing test cases pass. I was not prepared for the profound ramifications that writing code this way would have. This practice has completely transformed the way I write software, and transformed it for the better. You can see that transformation in this book. Some of the code written in this book was written before 1999. You won't find test cases for that code. On the other hand, all of the code written after 1999 is presented with test cases, and the test cases are typically presented first. I'm sure you'll note the difference.

So, by the fall of 1999 I was convinced that Object Mentor should adopt XP as its process of choice and that I should let go of my desire to write my own process. Kent had done an excellent job of articulating the practices and process of XP, and my own feeble attempts paled in comparison.

2. `http://c2.com/cgi/wiki`. This website contains a vast number of articles on an immense variety of subjects. Its authors number in the hundreds or thousands. It has been said that only Ward Cunningham could instigate a social revolution using a few lines of Perl.

3. Any credible intellectual work written between 1995 and 2001 must use the term "Kuhnian." It refers to the book, *The Structure of Scientific Revolutions*, by Thomas S. Kuhn, The University of Chicago Press, 1962.

4. If you mention Kuhn twice in a paper, you get extra credit.

Organization

This book is organized into six major sections followed by several appendices.

- Section 1: *Agile Development.*
  This section describes the concept of agile development. It starts with the Manifesto of the Agile Alliance, provides an overview of Extreme Programming (XP), and then goes into many small case studies that illuminate some of the individual XP practices—especially those that have an impact upon the way we design and write code.
- Section 2: *Agile Design*
  The chapters in this section talk about object-oriented software design. The first chapter asks the question, *What is Design?* It discusses the problem of, and techniques for, managing complexity. Finally, the section culminates with the *principles of object-oriented class design.*
- Section 3: *The Payroll Case Study*
  This is the largest and most complete case study in the book. It describes the object-oriented design and C++ implementation of a simple batch payroll system. The first few chapters in this section describe the design patterns that the case study encounters. The final two chapters contain the full case study.
- Section 4: *Packaging the Payroll System*
  This section begins by describing the *principles of object-oriented package design.* It then goes on to illustrate those principles by incrementally packaging the classes from the previous section.
- Section 5: *The Weather Station Case Study*
  This section contains one of the case studies that was originally planned for the Booch book. The Weather Station study describes a company that has made a significant business decision and explains how the Java development team responds to it. As usual, the section begins with a description of the design patterns that will be used and then culminates in the description of the design and implementation.
- Section 6: *The ETS Case Study*
  This section contains a description of an actual project that the author participated in. This project has been in production since 1999. It is the automated test system used to deliver and score the registry examination for the National Council of Architectural Registration Boards.
- UML Notation Appendices
  The first two appendices contains several small case studies that are used to describe the UML notation.
- Miscellaneous Appendices

How to Use This Book

If You are a Developer...

Read the book cover to cover. This book was written primarily for developers, and it contains the information you need to develop software in an agile manner. Reading the book cover to cover introduces practices, then principles, then patterns, and then it provides case studies that tie them all together. Integrating all this knowledge will help you get your projects *done*.

If You Are a Manager or Business Analyst...

Read Section 1, *Agile Development.* The chapters in this section provide an in-depth discussion of agile principles and practices. They'll take you from requirements to planning to testing, refactoring, and programming. It will give you guidance on how to build teams and manage projects. It will help you get your projects *done*.

If You Want to Learn UML...

First read Appendix A, *UML Notation I: The CGI Example.* Then read Appendix B, *UML Notation II: The STATMUX.* Then, read all the chapters in Section 3, *The Payroll Case Study.* This course of reading will give you a good grounding in both the syntax and use of UML. It will also help you translate between UML and a programming language like Java or C++.

If You Want to Learn Design Patterns...

To find a particular pattern, use the "List of Design Patterns" on page xxii to find the pattern you are interested in.

To learn about patterns in general, read Section 2, *Agile Design* to first learn about design principles, and then read Section 3, *The Payroll Case Study*; Section 4, *Packaging the Payroll System*; Section 5, *The Weather Station Case Study*; and Section 6, *The ETS Case Study.* These sections define all the patterns and show how to use them in typical situations.

If You Want to Learn about Object-Oriented Design Principles...

Read Section 2, *Agile Design*; Section 3, *The Payroll Case Study*; and Section 4, *Packaging the Payroll System.* These chapters will describe the principles of object-oriented design and will show you how to use them.

If You Want to Learn about Agile Development Methods...

Read Section 1, *Agile Development.* This section describes agile development from requirements to planning, testing, refactoring, and programming.

If You Want a Chuckle or Two...

Read Appendix C, *A Satire of Two Companies.*

Acknowledgments

A heartfelt thanks to:

Lowell Lindstrom, Brian Button, Erik Meade, Mike Hill, Michael Feathers, Jim Newkirk, Micah Martin, Angelique Thouvenin Martin, Susan Rosso, Talisha Jefferson, Ron Jeffries, Kent Beck, Jeff Langr, David Farber, Bob Koss, James Grenning, Lance Welter, Pascal Roy, Martin Fowler, John Goodsen, Alan Apt, Paul Hodgetts, Phil Markgraf, Pete McBreen, H. S. Lahman, Dave Harris, James Kanze, Mark Webster, Chris Biegay, Alan Francis, Fran Daniele, Patrick Lindner, Jake Warde, Amy Todd, Laura Steele, William Pietr, Camille Trentacoste, Vince O'Brien, Gregory Dulles, Lynda Castillo, Craig Larman, Tim Ottinger, Chris Lopez, Phil Goodwin, Charles Toland, Robert Evans, John Roth, Debbie Utley, John Brewer, Russ Ruter, David Vydra, Ian Smith, Eric Evans, everyone in the Silicon Valley Patterns group, Pete Brittingham, Graham Perkins, Phlip, and Richard MacDonald.

The books reviewers:

Pete McBreen / McBreen Consulting
Stephen J. Mellor / Projtech.com
Brian Button / Object Mentor Inc.
Bjarne Stroustrup / AT & T Research
Micah Martin / Object Mentor Inc.
James Grenning / Object Mentor Inc.

A very special thanks to Grady Booch and Paul Becker for allowing me to include chapters that were originally slated for Grady's third edition of *Object Oriented Analysis and Design with Applications.*

A special thanks to Jack Reeves for graciously allowing me to reproduce his "What is Design?" article.

Another special thanks to Erich Gamma, for writing the forward to this book. I hope the fonts are better this time Erich!

The wonderful and sometimes dazzling illustrations at the head of each chapter were drawn by Jennifer Kohnke. The decorative illustrations scattered throughout the midst of the chapters are the lovely product of Angela Dawn Martin Brooks, my daughter, and one of the joys of my life.

Resources

All the source code in this book can be downloaded from `www.objectmentor.com/PPP`.

About the Authors

Robert C. Martin

Robert C. Martin (Uncle Bob) has been a software professional since 1970 and an international software consultant since 1990. He is founder and president of Object Mentor Inc., a team of experienced consultants who mentor their clients worldwide in the fields of C++, Java, .NET, OO, Patterns, UML, Agile Methodologies, and Extreme Programming. In 1995, Robert authored the best-selling book: *Designing Object Oriented C++ Applications using the Booch Method*, published by Prentice Hall. From 1996 to 1999 he was the editor-in-chief of the C++ Report. In 1997, he was chief editor of the book: *Pattern Languages of Program Design 3*, published by Addison–Wesley. In 1999, he was the editor of *More C++ Gems* published by Cambridge Press. He is co-author, with James Newkirk, of *XP in Practice*, Addision–Wesley, 2001. In 2002, he wrote the long awaited *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2002. He has published dozens of articles in various trade journals, and is a regular speaker at international conferences and trade shows. And he's as happy as a clam.

James W. Newkirk

James Newkirk is a Software Development Manager/Architect. His eighteen years of experience ranges from programming real-time micro-controllers to web services. He co-wrote *Extreme Programming in Practice*, published by Addison–Wesley, 2001. Since August of 2000 he has been working with the .NET Framework and has contributed to the development of NUnit, a unit-testing tool for .NET.

Robert S. Koss

Robert S. Koss, Ph.D., has been writing software for 29 years. He has applied the principles of Object Oriented Design to many projects where he has served in roles ranging from programmer to senior architect. Dr. Koss has taught hundreds of OOD and programming language courses to thousands of students throughout the world. He is currently employed as a Senior Consultant at Object Mentor, Inc.

Brief Contents

Section 1	**Agile Development**	**1**
Chapter 1	**Agile Practices**	**3**
Chapter 2	**Overview of Extreme Programming**	**11**
Chapter 3	**Planning**	**19**
Chapter 4	**Testing**	**23**
Chapter 5	**Refactoring**	**31**
Chapter 6	**A Programming Episode**	**43**
Section 2	**Agile Design**	**85**
Chapter 7	**What Is Agile Design?**	**87**
Chapter 8	**SRP: The Single-Responsibility Principle**	**95**
Chapter 9	**OCP: The Open–Closed Principle**	**99**
Chapter 10	**LSP: The Liskov Substitution Principle**	**111**
Chapter 11	**DIP: The Dependency-Inversion Principle**	**127**
Chapter 12	**ISP: The Interface-Segregation Principle**	**135**
Section 3	**The Payroll Case Study**	**147**
Chapter 13	**COMMAND and ACTIVE OBJECT**	**151**
Chapter 14	**TEMPLATE METHOD & STRATEGY: Inheritance vs. Delegation**	**161**
Chapter 15	**FACADE and MEDIATOR**	**173**
Chapter 16	**SINGLETON and MONOSTATE**	**177**

Chapter 17 NULL OBJECT 189
Chapter 18 The Payroll Case Study: Iteration One Begins 193
Chapter 19 The Payroll Case Study: Implementation 205

Section 4 Packaging the Payroll System 251

Chapter 20 Principles of Package Design 253
Chapter 21 FACTORY 269
Chapter 22 The Payroll Case Study (Part 2) 275

Section 5 The Weather Station Case Study 291

Chapter 23 COMPOSITE 293
Chapter 24 OBSERVER—Backing into a Pattern 297
Chapter 25 ABSTRACT SERVER, ADAPTER, and BRIDGE 317
Chapter 26 PROXY and STAIRWAY TO HEAVEN: Managing Third Party APIs 327
Chapter 27 Case Study: Weather Station 355

Section 6 The ETS Case Study 385

Chapter 28 VISITOR 387
Chapter 29 STATE 419
Chapter 30 The ETS Framework 443
Appendix A UML Notation I: The CGI Example 467
Appendix B UML Notation II: The STATMUX 489
Appendix C A Satire of Two Companies 507
Appendix D The Source Code Is the Design 517

Index 525

Contents

Foreword iii

Preface iv

About the Authors ix

List of Design Patterns xxii

Section 1 **Agile Development** 1

Chapter 1 **Agile Practices** 3

The Agile Alliance 4

The Manifesto of the Agile Alliance 4

Principles 6

Conclusion 8

Bibliography 9

Chapter 2 **Overview of Extreme Programming** 11

The Practices of Extreme Programming 11

Customer Team Member 11

User Stories 12

Short Cycles 12

Acceptance Tests 13

Pair Programming 13

Test-Driven Development 14

Collective Ownership 14

Continuous Integration 14

Sustainable Pace 15

Open Workspace 15

The Planning Game 15

Simple Design 15

Refactoring 16

Metaphor 16

Conclusion 17

Bibliography 17

Chapter 3 **Planning** **19**
Initial Exploration 20
Spiking, Splitting, and Velocity 20
Release Planning 20
Iteration Planning 21
Task Planning 21
The Halfway Point 22
Iterating 22
Conclusion 22
Bibliography 22

Chapter 4 **Testing** **23**
Test Driven Development 23
An Example of Test-First Design 24
Test Isolation 25
Serendipitous Decoupling 26
Acceptance Tests 27
Example of Acceptance Testing 27
Serendipitous Architecture 29
Conclusion 29
Bibliography 29

Chapter 5 **Refactoring** **31**
Generating Primes: A Simple Example of Refactoring 32
The Final Reread 38
Conclusion 42
Bibliography 42

Chapter 6 **A Programming Episode** **43**
The Bowling Game 44
Conclusion 82

Section 2 **Agile Design** **85**
Symptoms of Poor Design 85
Principles 86
Smells and Principles 86
Bibliography 86

Chapter 7 **What Is Agile Design?** **87**
What Goes Wrong with Software? 87
Design Smells—The Odors of Rotting Software 88
What Stimulates the Software to Rot? 89
Agile Teams Don't Allow the Software to Rot 90
The "Copy" Program 90
Agile Design of the `Copy` Example 93
How Did the Agile Developers Know What to Do? 94
Keeping the Design As Good As It Can Be 94
Conclusion 94
Bibliography 94

Chapter 8 **SRP: The Single-Responsibility Principle** **95**

A CLASS SHOULD HAVE ONLY ONE REASON TO CHANGE.

SRP: The Single-Responsibility Principle 95
What Is a Responsibility? 97
Separating Coupled Responsibilities 97
Persistence 98
Conclusion 98
Bibliography 98

Chapter 9 **OCP: The Open–Closed Principle** **99**

SOFTWARE ENTITIES (CLASSES, MODULES, FUNCTIONS, ETC.) SHOULD BE OPEN FOR EXTENSION, BUT CLOSED FOR MODIFICATION.

OCP: The Open–Closed Principle 99
Description 100
Abstraction Is the Key 100
The `Shape` Application 101
Violating the OCP 101
Conforming to the OCP 103
OK, I Lied 104
Anticipation and "Natural" Structure 105
Putting the "Hooks" In 105
Using Abstraction to Gain Explicit Closure 106
Using a "Data-Driven" Approach to Achieve Closure 107
Conclusion 108
Bibliography 109

Chapter 10 **LSP: The Liskov Substitution Principle** **111**

SUBTYPES MUST BE SUBSTITUTABLE FOR THEIR BASE TYPES.

LSP: The Liskov Substitution Principle 111
A Simple Example of a Violation of the LSP 112
Square and Rectangle, a More Subtle Violation 113
The Real Problem 115
Validity Is Not Intrinsic 116
ISA Is about Behavior 116
Design by Contract 117
Specifying Contracts in Unit Tests 117
A Real Example 117
Motivation 118
Problem 119
A Solution That Does *Not* Conform to the LSP 120
An LSP-Compliant Solution 120
Factoring Instead of Deriving 121
Heuristics and Conventions 124
Degenerate Functions in Derivatives 124
Throwing Exceptions from Derivatives 124
Conclusion 125
Bibliography 125

Chapter 11 **DIP: The Dependency-Inversion Principle** **127**

A. HIGH-LEVEL MODULES SHOULD NOT DEPEND UPON LOW-LEVEL MODULES. BOTH SHOULD DEPEND ON ABSTRACTIONS.

B. ABSTRACTIONS SHOULD NOT DEPEND ON DETAILS. DETAILS SHOULD DEPEND ON ABSTRACTIONS.

DIP: The Dependency-Inversion Principle 127
Layering 128
An Inversion of Ownership 128
Depend on Abstractions 129
A Simple Example 130
Finding the Underlying Abstraction 131
The Furnace Example 132
Dynamic v. Static Polymorphism 133
Conclusion 134
Bibliography 134

Chapter 12 **ISP: The Interface-Segregation Principle** **135**

Interface Pollution 135
Separate Clients Mean Separate Interfaces 137
The Backwards Force Applied by Clients Upon Interfaces 137

CLIENTS SHOULD NOT BE FORCED TO DEPEND ON METHODS THAT THEY DO NOT USE.

ISP: The Interface-Segregation Principle 137
Class Interfaces v. Object Interfaces 138
Separation through Delegation 138
Separation through Multiple Inheritance 139
The ATM User Interface Example 139
The Polyad v. the Monad 144
Conclusion 145
Bibliography 145

Section 3 **The Payroll Case Study** **147**

Rudimentary Specification of the Payroll System 148
Exercise 148
Use Case 1: Add New Employee 148
Use Case 2: Deleting an Employee 149
Use Case 3: Post a `Time Card` 149
Use Case 4: Posting a `Sales Receipt` 149
Use Case 5: Posting a Union Service Charge 150
Use Case 6: Changing Employee Details 150
Use Case 7: Run the Payroll for Today 150

Chapter 13 **COMMAND and ACTIVE OBJECT** **151**

Simple Commands 152
Transactions 153
Physical and Temporal Decoupling 154
Temporal Decoupling 154
UNDO 154

ACTIVE OBJECT 155
Conclusion 159
Bibliography 159

Chapter 14 **TEMPLATE METHOD & STRATEGY: Inheritance vs. Delegation** **161**
TEMPLATE METHOD 162
Pattern Abuse 164
Bubble Sort 165
STRATEGY 168
Sorting Again 170
Conclusion 172
Bibliography 172

Chapter 15 **FACADE and MEDIATOR** **173**
FACADE 173
MEDIATOR 174
Conclusion 176
Bibliography 176

Chapter 16 **SINGLETON and MONOSTATE** **177**
SINGLETON 178
Benefits of the SINGLETON 179
Costs of the SINGLETON 179
SINGLETON in Action 179
MONOSTATE 180
Benefits of MONOSTATE 182
Costs of MONOSTATE 182
MONOSTATE in Action 182
Conclusion 187
Bibliography 187

Chapter 17 **NULL OBJECT** **189**
Conclusion 192
Bibliography 192

Chapter 18 **The Payroll Case Study: Iteration One Begins** **193**
Introduction 193
Specification 193
Analysis by Use Cases 194
Adding Employees 195
Deleting Employees 196
Posting Time Cards 196
Posting Sales Receipts 197
Posting a Union Service Charge 197
Changing Employee Details 198
Payday 199
Reflection: What Have We Learned? 201
Finding the Underlying Abstractions 201
The Schedule Abstraction 201

Payment Methods 202
Affiliations 202
Conclusion 203
Bibliography 203

Chapter 19 **The Payroll Case Study: Implementation 205**
Adding Employees 206
The Payroll Database 207
Using TEMPLATE METHOD to Add Employees 209
Deleting Employees 212
Global Variables 213
Time Cards, Sales Receipts, and Service Charges 214
Changing Employees 220
Changing Classification 224
What Was I Smoking? 229
Paying Employees 233
Do We Want Developers Making Business Decisions? 235
Paying Salaried Employees 235
Paying Hourly Employees 237
Pay Periods: A Design Problem 241
Main Program 248
The Database 248
Summary of Payroll Design 249
History 249
Resources 250
Bibliography 250

Section 4 **Packaging the Payroll System 251**

Chapter 20 **Principles of Package Design 253**
Designing with Packages? 253
Granularity: The Principles of Package Cohesion 254
The Reuse–Release Equivalence Principle (REP) 254
THE GRANULE OF REUSE IS THE GRANULE OF RELEASE.
The Common-Reuse Principle (CRP) 255
THE CLASSES IN A PACKAGE ARE REUSED TOGETHER. IF YOU REUSE ONE OF THE CLASSES IN A PACKAGE, YOU REUSE THEM ALL.
The Common-Closure Principle (CCP) 256
THE CLASSES IN A PACKAGE SHOULD BE CLOSED TOGETHER AGAINST THE SAME KINDS OF CHANGES. A CHANGE THAT AFFECTS A PACKAGE AFFECTS ALL THE CLASSES IN THAT PACKAGE AND NO OTHER PACKAGES.
Summary of Package Cohesion 256
Stability: The Principles of Package Coupling 256
The Acyclic-Dependencies Principle (ADP) 256
ALLOW NO CYCLES IN THE PACKAGE DEPENDENCY GRAPH.
The Weekly Build 257
Eliminating Dependency Cycles 257
The Effect of a Cycle in the Package Dependency Graph 258

Breaking the Cycle 259
The "Jitters" 259
Top-Down Design 260
The Stable-Dependencies Principle (SDP) 261
DEPEND IN THE DIRECTION OF STABILITY.
Stability 261
Stability Metrics 262
Not All Packages Should Be Stable 263
Where Do We Put the High-level Design? 264
The Stable-Abstractions Principle (SAP) 264
A PACKAGE SHOULD BE AS ABSTRACT AS IT IS STABLE.
Measuring Abstraction 265
The Main Sequence 265
Distance from the Main Sequence 266
Conclusion 268

Chapter 21 **FACTORY** **269**
A Dependency Cycle 271
Substitutable Factories 272
Using Factories for Test Fixtures 273
How Important Is It to Use Factories? 274
Conclusion 274
Bibliography 274

Chapter 22 **The Payroll Case Study (Part 2)** **275**
Package Structure and Notation 276
Applying the Common Closure Principle (CCP) 277
Applying the Reuse–Release Equivalency Principle (REP) 278
Coupling and Encapsulation 279
Metrics 281
Applying the Metrics to the Payroll Application 282
Object Factories 285
The Object Factory for `TransactionImplementation` 286
Initializing the Factories 286
Rethinking the Cohesion Boundaries 287
The Final Package Structure 287
Conclusion 290
Bibliography 290

Section 5 **The Weather Station Case Study** **291**

Chapter 23 **COMPOSITE** **293**
Example: Composite Commands 294
Multiplicity or Not Multiplicity 295

Chapter 24 **OBSERVER—Backing into a Pattern** **297**
The Digital Clock 297

Conclusion 314
The Use of Diagrams in this Chapter 314
The OBSERVER Pattern 315
How OBSERVER Manages the Principles of OOD 316
Bibliography 316

Chapter 25 **ABSTRACT SERVER, ADAPTER, and BRIDGE** **317**

ABSTRACT SERVER 318
Who Owns the Interface? 318
Adapter 319
The Class Form of ADAPTER 319
The Modem Problem, ADAPTERS and LSP 320
BRIDGE 322
Conclusion 324
Bibliography 325

Chapter 26 **PROXY and STAIRWAY TO HEAVEN: Managing Third Party APIs** **327**

PROXY 327
Proxifying the Shopping Cart 332
Summary of PROXY 344
Dealing with Databases, Middleware, and Other Third Party Interfaces 345
STAIRWAY TO HEAVEN 347
Example of STAIRWAY TO HEAVEN 348
Other Patterns That Can Be Used with Databases 353
Conclusion 354
Bibliography 354

Chapter 27 **Case Study: Weather Station** **355**

The Cloud Company 355
The WMS-LC Software 356
Language Selection 357
Nimbus-LC Software Design 357
24-Hour History and Persistence 368
Implementing the HiLo Algorithms 371
Conclusion 379
Bibliography 379
Nimbus-LC Requirements Overview 379
Usage Requirements 379
24-Hour History 379
User Setup 379
Administrative Requirements 380
Nimbus-LC Use Cases 380
Actors 380
Use Cases 380
Measurement History 380
Setup 381
Administration 381
Nimbus-LC Release Plan 381
Introduction 381
Release I 381

Risks 382
Deliverable(s) 382
Release II 382
Use Cases Implemented 382
Risks 383
Deliverable(s) 383
Release III 383
Use Cases Implemented 383
Risks 383
Deliverable(s) 383

Section 6 **The ETS Case Study** **385**

Chapter 28 **VISITOR** **387**
The VISITOR Family of Design Patterns 388
VISITOR 388
VISITOR is Like a Matrix 391
ACYCLIC VISITOR 391
ACYCLIC VISITOR Is Like a Sparse Matrix 396
Using VISITOR in Report Generators 396
Other Uses of VISITOR 402
DECORATOR 403
Multiple Decorators 406
EXTENSION OBJECT 408
Conclusion 418
Reminder 418
Bibliography 418

Chapter 29 **STATE** **419**
Overview of Finite State Automata 419
Implementation Techniques 421
Nested Switch/Case Statements 421
Interpreting Transition Tables 424
The STATE Pattern 426
SMC—The State-Machine Compiler 429
Where Should State Machines be Used? 432
High-Level Application Policies for GUIs 432
GUI Interaction Controllers 433
Distributed Processing 433
Conclusion 434
Listings 434
`Turnstile.java` Using Table Interpretation 434
`Turnstile.java` Generated by SMC, and Other Support Files 437
Bibliography 441

Chapter 30 **The ETS Framework** **443**
Introduction 443
Project Overview 443

Early History 1993–1994 445
Framework? 445
Framework! 446
The 1994 Team 446
The Deadline 446
The Strategy 446
Results 447
Framework Design 448
The Common Requirements of the Scoring Applications 448
The Design of the Scoring Framework 450
A Case for TEMPLATE METHOD 453
Write a Loop Once 454
The Common Requirements of the Delivery Applications 456
The Design of the Delivery Framework 457
The Taskmaster Architecture 462
Conclusion 465
Bibliography 466

Appendix A **UML Notation I: The CGI Example** **467**

Course Enrollment System: Problem Description 468
Actors 469
Use Cases 469
The Domain Model 472
The Architecture 476
Abstract Classes and Interfaces in Sequence Diagrams 485
Summary 486
Bibliography 487

Appendix B **UML Notation II: The STATMUX** **489**

The Statistical Multiplexor Definition 489
The Software Environment 490
The Real-time Constraints 490
The Input Interrupt Service Routine 491
The Output Service Interrupt Routine 495
The Communications Protocol 496
Conclusion 506
Bibliography 506

Appendix C **A Satire of Two Companies** **507**

Rufus, Inc.
Project Kickoff 507
Rupert Industries
Project: ~Alpha~ 507

Appendix D **The Source Code Is the Design** **517**

What Is Software Design? 517
Afterword 523

Index **525**

List of Design Patterns

Abstract Server	318
Active Object	155
Acyclic Visitor	391
Adapter	319
Bridge	322
Command	151
Composite	293
Decorator	403
Extension Object	408
Facade	173
Factory	269
Mediator	174
Monostate	180
Null Object	189
Observer	315
Proxy	327
Singleton	178
Stairway to Heaven	347
State	426
Strategy	168
Taskmaster	462
Template Method	162
Visitor	388

SECTION 1

Agile Development



"Human Interactions are complicated and never very crisp and clean in their effects, but they matter more than any other aspect of the work."

—Tom DeMarco and Timothy Lister
Peopleware, p. 5

Principles, patterns, and practices are important, but it's the people that make them work. As Alistair Cockburn says,[1] "Process and technology are a second-order effect on the outcome of a project. The first-order effect is the people."

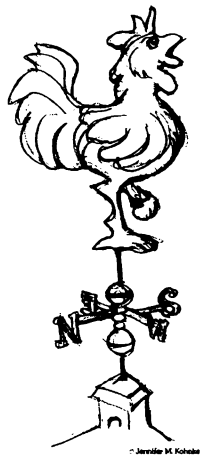
We cannot manage teams of programmers as if they were systems made up of components driven by a process. People are not "plug-compatible programming units."[2] If our projects are to succeed, we are going to have to build collaborative and self-organizing teams.

Those companies that encourage the formation of such teams will have a *huge* competitive advantage over those who hold the view that a software-development organization is nothing more than a pile of twisty little people all alike. A gelled software team is the most powerful software-development force there is.

1. Private communication.
2. A term coined by Kent Beck.

1

Agile Practices



The weather-cock on the church spire, though made of iron, would soon be broken by the storm-wind if it did not understand the noble art of turning to every wind.

—Heinrich Heine

Many of us have lived through the nightmare of a project with no practices to guide it. The lack of effective practices leads to unpredictability, repeated error, and wasted effort. Customers are disappointed by slipping schedules, growing budgets, and poor quality. Developers are disheartened by working ever longer hours to produce ever poorer software.

Once we have experienced such a fiasco, we become afraid of repeating the experience. Our fears motivate us to create a *process* that constrains our activities and demands certain outputs and artifacts. We draw these constraints and outputs from past experience, choosing things that appeared to work well in previous projects. Our hope is that they will work again and take away our fears.

However, projects are not so simple that a few constraints and artifacts can reliably prevent error. As errors continue to be made, we diagnose those errors and put in place even more constraints and artifacts in order to prevent those errors in the future. After many, projects we may find ourselves overloaded with a huge cumbersome process that greatly impedes our ability to get anything done.

A big cumbersome process can create the very problems that it is designed to prevent. It can slow the team to the extent that schedules slip and budgets bloat. It can reduce responsiveness of the team to the point where they

are always creating the wrong product. Unfortunately, this leads many teams to believe that they don't have enough process. So, in a kind of runaway-process inflation, they make their process ever larger.

Runaway-process inflation is a good description of the state of affairs in many software companies circa 2000 A.D. Though there were still many teams operating without a process, the adoption of very large, heavyweight processes is rapidly growing, especially in large corporations. (See Appendix C.)

The Agile Alliance

In early 2001, motivated by the observation that software teams in many corporations were stuck in a quagmire of ever-increasing process, a group of industry experts met to outline the values and principles that would allow software teams to work quickly and respond to change. They called themselves the *Agile Alliance*.[1] Over the next several months, they worked to create a statement of values. The result was *The Manifesto of the Agile Alliance*.

The Manifesto of the Agile Alliance

Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck	Mike Beedle	Arie van Bennekum	Alistair Cockburn
Ward Cunningham	Martin Fowler	James Grenning	Jim Highsmith
Andrew Hunt	Ron Jeffries	Jon Kern	Brian Marick
Robert C. Martin	Steve Mellor	Ken Schwaber	Jeff Sutherland
Dave Thomas			

Individuals and interactions over processes and tools. People are the most important ingredient of success. A good process will not save the project from failure if the team doesn't have strong players, but a bad process can make even the strongest of players ineffective. Even a group of strong players can fail badly if they don't work as a team.

A strong player is not necessarily an ace programmer. A strong player may be an average programmer, but someone who works well with others. Working well with others, communicating and interacting, is more important than raw programming talent. A team of average programmers who communicate well are more likely to succeed than a group of superstars who fail to interact as a team.

The right tools can be very important to success. Compilers, IDEs, source-code control systems, etc. are all vital to the proper functioning of a team of developers. However, tools can be overemphasized. An overabundance of big, unwieldy tools is just as bad as a lack of tools.

My advice is to start small. Don't assume you've outgrown a tool until you've tried it and found you can't use it. Instead of buying the top-of-the-line, megaexpensive, source-code control system, find a free one and use it

1. agilealliance.org

until you can demonstrate that you've outgrown it. Before you buy team licenses for the best of all CASE tools, use white boards and graph paper until you can reasonably show that you need more. Before you commit to the top-shelf behemoth database system, try flat files. Don't assume that bigger and better tools will automatically help you do better. Often they hinder more than they help.

Remember, building the team is more important than building the environment. Many teams and managers make the mistake of building the environment first and expecting the team to gel automatically. Instead, work to create the team, and then let the team configure the environment on the basis of need.

Working software over comprehensive documentation. Software without documentation is a disaster. Code is not the ideal medium for communicating the rationale and structure of a system. Rather, the team needs to produce human-readable documents that describe the system and the rationale for their design decisions.

However, too much documentation is worse than too little. Huge software documents take a great deal of time to produce and even more time to keep in sync with the code. If they are not kept in sync, then they turn into large, complicated lies and become a significant source of misdirection.

It is always a good idea for the team to write and maintain a rationale and structure document, but that document needs to be *short* and *salient*. By "short," I mean one or two dozen pages at most. By "salient," I mean it should discuss the overall design rationale and only the highest-level structures in the system.

If all we have is a short rationale and structure document, how do we train new team members to work on the system? We work closely with them. We transfer our knowledge to them by sitting next to them and helping them. We make them part of the team through close training and interaction.

The two documents that are the best at transferring information to new team members are the code and the team. The code does not lie about what it does. It may be hard to extract rationale and intent from the code, but the code is the only unambiguous source of information. The team members hold the ever-changing road map of the system in their heads. There is no faster and more efficient way to transfer that road map to others than human-to-human interaction.

Many teams have gotten hung up in the pursuit of documentation instead of software. This is often a fatal flaw. There is a simple rule called **Martin's first law of documentation** that prevents it:

> *Produce no document unless its need is immediate and significant.*

Customer collaboration over contract negotiation. Software cannot be ordered like a commodity. You cannot write a description of the software you want and then have someone develop it on a fixed schedule for a fixed price. Time and time again, attempts to treat software projects in this manner have failed. Sometimes the failures are spectacular.

It is tempting for the managers of a company to tell their development staff what their needs are, and then expect that staff to go away for a while and return with a system that satisfies those needs. However, this mode of operation leads to poor quality and failure.

Successful projects involve customer feedback on a regular and frequent basis. Rather than depending on a contract or a statement of work, the customer of the software works closely with the development team, providing frequent feedback on their efforts.

A contract that specifies the requirements, schedule, and cost of a project is fundamentally flawed. In most cases, the terms it specifies become meaningless long before the project is complete.[2] The best contracts are those that govern the way the development team and the customer will work together.

As an example of a successful contract, take one I negotiated in 1994 for a large, multiyear, half-million-line project. We, the development team, were paid a relatively low monthly rate. Large payouts were made to us when we delivered certain large blocks of functionality. Those blocks were not specified in detail by the contract. Rather,

2. Sometimes long before the contract is signed!